If the counter starts at 0, then $\Phi(D_0) = 0$. Since $\Phi(D_i) \geq 0$ for all i, the total amortized cost of a sequence of n INCREMENT operations is an upper bound on the total actual cost, and so the worst-case cost of n INCREMENT operations is $O(n)$.

The potential method provides a simple and clever way to analyze the counter even when it does not start at 0. The counter starts with b_0 1-bits, and after n INCREMENT operations it has b_n 1-bits, where $0 \leq b_0, b_n \leq k$. Rewrite equation (16.3) as

$$\sum_{i=1}^{n} c_i = \sum_{i=1}^{n} \hat{c}_i - \Phi(D_n) + \Phi(D_0) .$$

Since $\Phi(D_0) = b_0$, $\Phi(D_n) = b_n$, and $\hat{c}_i \leq 2$ for all $1 \leq i \leq n$, the total actual cost of n INCREMENT operations is

$$\begin{aligned} \sum_{i=1}^{n} c_i &\leq \sum_{i=1}^{n} 2 - b_n + b_0 \\ &= 2n - b_n + b_0 . \end{aligned}$$

In particular, $b_0 \leq k$ means that as long as $k = O(n)$, the total actual cost is $O(n)$. In other words, if at least $n = \Omega(k)$ INCREMENT operations occur, the total actual cost is $O(n)$, no matter what initial value the counter contains.

Exercises

16.3-1

Suppose you have a potential function Φ such that $\Phi(D_i) \geq \Phi(D_0)$ for all i, but $\Phi(D_0) \neq 0$. Show that there exists a potential function Φ' such that $\Phi'(D_0) = 0$, $\Phi'(D_i) \geq 0$ for all $i \geq 1$, and the amortized costs using Φ' are the same as the amortized costs using Φ.

16.3-2

Redo Exercise 16.1-3 using a potential method of analysis.

16.3-3

Consider an ordinary binary min-heap data structure supporting the instructions INSERT and EXTRACT-MIN that, when there are n items in the heap, implements each operation in $O(\lg n)$ worst-case time. Give a potential function Φ such that the amortized cost of INSERT is $O(\lg n)$ and the amortized cost of EXTRACT-MIN is $O(1)$, and show that your potential function yields these amortized time bounds. Note that in the analysis, n is the number of items currently in the heap, and you do not know a bound on the maximum number of items that can ever be stored in the heap.

16.3-4
What is the total cost of executing n of the stack operations PUSH, POP, and MULTIPOP, assuming that the stack begins with s_0 objects and finishes with s_n objects?

16.3-5
Show how to implement a queue with two ordinary stacks (Exercise 10.1-7) so that the amortized cost of each ENQUEUE and each DEQUEUE operation is $O(1)$.

16.3-6
Design a data structure to support the following two operations for a dynamic multiset S of integers, which allows duplicate values:

INSERT(S, x) inserts x into S.

DELETE-LARGER-HALF(S) deletes the largest $\lceil |S| / 2 \rceil$ elements from S.

Explain how to implement this data structure so that any sequence of m INSERT and DELETE-LARGER-HALF operations runs in $O(m)$ time. Your implementation should also include a way to output the elements of S in $O(|S|)$ time.

16.4 Dynamic tables

When you design an application that uses a table, you do not always know in advance how many items the table will hold. You might allocate space for the table, only to find out later that it is not enough. The program must then reallocate the table with a larger size and copy all items stored in the original table over into the new, larger table. Similarly, if many items have been deleted from the table, it might be worthwhile to reallocate the table with a smaller size. This section studies this problem of dynamically expanding and contracting a table. Amortized analyses will show that the amortized cost of insertion and deletion is only $O(1)$, even though the actual cost of an operation is large when it triggers an expansion or a contraction. Moreover, you'll see how to guarantee that the unused space in a dynamic table never exceeds a constant fraction of the total space.

Let's assume that the dynamic table supports the operations TABLE-INSERT and TABLE-DELETE. TABLE-INSERT inserts into the table an item that occupies a single ***slot***, that is, a space for one item. Likewise, TABLE-DELETE removes an item from the table, thereby freeing a slot. The details of the data-structuring method used to organize the table are unimportant: it could be a stack (Section 10.1.3), a heap (Chapter 6), a hash table (Chapter 11), or something else.

It is convenient to use a concept introduced in Section 11.2, where we analyzed hashing. The ***load factor*** $\alpha(T)$ of a nonempty table T is defined as the number of items stored in the table divided by the size (number of slots) of the table. An empty table (one with no slots) has size 0, and its load factor is defined to be 1. If the load factor of a dynamic table is bounded below by a constant, the unused space in the table is never more than a constant fraction of the total amount of space.

We start by analyzing a dynamic table that allows only insertion and then move on to the more general case that supports both insertion and deletion.

16.4.1 Table expansion

Let's assume that storage for a table is allocated as an array of slots. A table fills up when all slots have been used or, equivalently, when its load factor is 1.[1] In some software environments, upon an attempt to insert an item into a full table, the only alternative is to abort with an error. The scenario in this section assumes, however, that the software environment, like many modern ones, provides a memory-management system that can allocate and free blocks of storage on request. Thus, upon inserting an item into a full table, the system can ***expand*** the table by allocating a new table with more slots than the old table had. Because the table must always reside in contiguous memory, the system must allocate a new array for the larger table and then copy items from the old table into the new table.

A common heuristic allocates a new table with twice as many slots as the old one. If the only table operations are insertions, then the load factor of the table is always at least $1/2$, and thus the amount of wasted space never exceeds half the total space in the table.

The TABLE-INSERT procedure on the following page assumes that T is an object representing the table. The attribute $T.table$ contains a pointer to the block of storage representing the table, $T.num$ contains the number of items in the table, and $T.size$ gives the total number of slots in the table. Initially, the table is empty: $T.num = T.size = 0$.

There are two types of insertion here: the TABLE-INSERT procedure itself and the ***elementary insertion*** into a table in lines 6 and 10. We can analyze the running time of TABLE-INSERT in terms of the number of elementary insertions by assigning a cost of 1 to each elementary insertion. In most computing environments, the overhead for allocating an initial table in line 2 is constant and the overhead for allocating and freeing storage in lines 5 and 7 is dominated by the cost of transfer-

[1] In some situations, such as an open-address hash table, it's better to consider a table to be full if its load factor equals some constant strictly less than 1. (See Exercise 16.4-2.)

```
TABLE-INSERT(T, x)
 1  if T.size == 0
 2      allocate T.table with 1 slot
 3      T.size = 1
 4  if T.num == T.size
 5      allocate new-table with 2 · T.size slots
 6      insert all items in T.table into new-table
 7      free T.table
 8      T.table = new-table
 9      T.size = 2 · T.size
10  insert x into T.table
11  T.num = T.num + 1
```

ring items in line 6. Thus, the actual running time of TABLE-INSERT is linear in the number of elementary insertions. An ***expansion*** occurs when lines 5–9 execute.

Now, we'll use all three amortized analysis techniques to analyze a sequence of n TABLE-INSERT operations on an initially empty table. First, we need to determine the actual cost c_i of the ith operation. If the current table has room for the new item (or if this is the first operation), then $c_i = 1$, since the only elementary insertion performed is the one in line 10. If the current table is full, however, and an expansion occurs, then $c_i = i$: the cost is 1 for the elementary insertion in line 10 plus $i - 1$ for the items copied from the old table to the new table in line 6. For n operations, the worst-case cost of an operation is $O(n)$, which leads to an upper bound of $O(n^2)$ on the total running time for n operations.

This bound is not tight, because the table rarely expands in the course of n TABLE-INSERT operations. Specifically, the ith operation causes an expansion only when $i - 1$ is an exact power of 2. The amortized cost of an operation is in fact $O(1)$, as an aggregate analysis shows. The cost of the ith operation is

$$c_i = \begin{cases} i & \text{if } i - 1 \text{ is an exact power of 2 ,} \\ 1 & \text{otherwise .} \end{cases}$$

The total cost of n TABLE-INSERT operations is therefore

$$\begin{aligned} \sum_{i=1}^{n} c_i &\le n + \sum_{j=0}^{\lfloor \lg n \rfloor} 2^j \\ &< n + 2n \qquad \text{(by equation (A.6) on page 1142)} \\ &= 3n \ , \end{aligned}$$

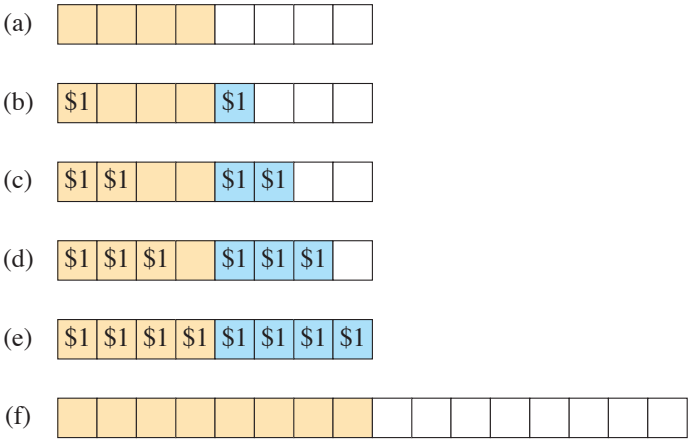

Figure 16.3 Analysis of table expansion by the accounting method. Each call of TABLE-INSERT charges \$3 as follows: \$1 to pay for the elementary insertion, \$1 on the item inserted as prepayment for it to be reinserted later, and \$1 on an item that was already in the table, also as prepayment for reinsertion. **(a)** The table immediately after an expansion, with 8 slots, 4 items (tan slots), and no stored credit. **(b)–(e)** After each of 4 calls to TABLE-INSERT, the table has one more item, with \$1 stored on the new item and \$1 stored on one of the 4 items that were present immediately after the expansion. Slots with these new items are blue. **(f)** Upon the next call to TABLE-INSERT, the table is full, and so it expands again. Each item had \$1 to pay for it to be reinserted. Now the table looks as it did in part (a), with no stored credit but 16 slots and 8 items.

because at most n operations cost 1 each and the costs of the remaining operations form a geometric series. Since the total cost of n TABLE-INSERT operations is bounded by $3n$, the amortized cost of a single operation is at most 3.

The accounting method can provide some intuition for why the amortized cost of a TABLE-INSERT operation should be 3. You can think of each item paying for three elementary insertions: inserting itself into the current table, moving itself the next time that the table expands, and moving some other item that was already in the table the next time that the table expands. For example, suppose that the size of the table is m immediately after an expansion, as shown in Figure 16.3 for $m = 8$. Then the table holds $m/2$ items, and it contains no credit. Each call of TABLE-INSERT charges \$3. The elementary insertion that occurs immediately costs \$1. Another \$1 resides on the item inserted as credit. The third \$1 resides as credit on one of the $m/2$ items already in the table. The table will not fill again until another $m/2 - 1$ items have been inserted, and thus, by the time the table contains m items and is full, each item has \$1 on it to pay for it to be reinserted it during the expansion.

Now, let's see how to use the potential method. We'll use it again in Section 16.4.2 to design a TABLE-DELETE operation that has an $O(1)$ amortized cost

as well. Just as the accounting method had no stored credit immediately after an expansion—that is, when $T.num = T.size/2$—let's define the potential to be 0 when $T.num = T.size/2$. As elementary insertions occur, the potential needs to increase enough to pay for all the reinsertions that will happen when the table next expands. The table fills after another $T.size/2$ calls of TABLE-INSERT, when $T.num = T.size$. The next call of TABLE-INSERT after these $T.size/2$ calls triggers an expansion with a cost of $T.size$ to reinsert all the items. Therefore, over the course of $T.size/2$ calls of TABLE-INSERT, the potential must increase from 0 to $T.size$. To achieve this increase, let's design the potential so that each call of TABLE-INSERT increases it by

$$\frac{T.size}{T.size/2} = 2 ,$$

until the table expands. You can see that the potential function

$$\Phi(T) = 2(T.num - T.size/2) \tag{16.4}$$

equals 0 immediately after the table expands, when $T.num = T.size/2$, and it increases by 2 upon each insertion until the table fills. Once the table fills, that is, when $T.num = T.size$, the potential $\Phi(T)$ equals $T.size$. The initial value of the potential is 0, and since the table is always at least half full, $T.num \geq T.size/2$, which implies that $\Phi(T)$ is always nonnegative. Thus, the sum of the amortized costs of n TABLE-INSERT operations gives an upper bound on the sum of the actual costs.

To analyze the amortized costs of table operations, it is convenient to think in terms of the change in potential due to each operation. Letting Φ_i denote the potential after the ith operation, we can rewrite equation (16.2) as

$$\begin{aligned} \hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= c_i + \Delta\Phi_i , \end{aligned}$$

where $\Delta\Phi_i$ is the change in potential due to the ith operation. First, consider the case when the ith insertion does not cause the table to expand. In this case, $\Delta\Phi_i$ is 2. Since the actual cost c_i is 1, the amortized cost is

$$\begin{aligned} \hat{c}_i &= c_i + \Delta\Phi_i \\ &= 1 + 2 \\ &= 3 . \end{aligned}$$

Now, consider the change in potential when the table does expand during the ith insertion because it was full immediately before the insertion. Let num_i denote the number of items stored in the table after the ith operation and $size_i$ denote the total size of the table after the ith operation, so that $size_{i-1} = num_{i-1} = i - 1$

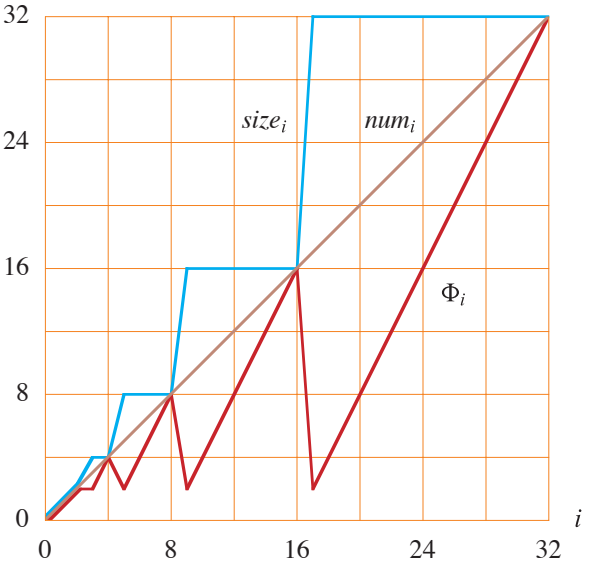

Figure 16.4 The effect of a sequence of n TABLE-INSERT operations on the number num_i of items in the table (the brown line), the number $size_i$ of slots in the table (the blue line), and the potential $\Phi_i = 2(num_i - size_i/2)$ (the red line), each being measured after the ith operation. Immediately before an expansion, the potential has built up to the number of items in the table, and therefore it can pay for moving all the items to the new table. Afterward, the potential drops to 0, but it immediately increases by 2 upon insertion of the item that caused the expansion.

and therefore $\Phi_{i-1} = 2(size_{i-1} - size_{i-1}/2) = size_{i-1} = i - 1$. Immediately after the expansion, the potential goes down to 0, and then the new item is inserted, causing the potential to increase to $\Phi_i = 2$. Thus, when the ith insertion triggers an expansion, $\Delta\Phi_i = 2 - (i - 1) = 3 - i$. When the table expands in the ith TABLE-INSERT operation, the actual cost c_i equals i (to reinsert $i - 1$ items and insert the ith item), giving an amortized cost of

$$\begin{aligned}\hat{c}_i &= c_i + \Delta\Phi_i \\ &= i + (3 - i) \\ &= 3 .\end{aligned}$$

Figure 16.4 plots the values of num_i, $size_i$, and Φ_i against i. Notice how the potential builds to pay for expanding the table.

16.4.2 Table expansion and contraction

To implement a TABLE-DELETE operation, it is simple enough to remove the specified item from the table. In order to limit the amount of wasted space, however, you might want to ***contract*** the table when the load factor becomes too small. Ta-

ble contraction is analogous to table expansion: when the number of items in the table drops too low, allocate a new, smaller table and then copy the items from the old table into the new one. You can then free the storage for the old table by returning it to the memory-management system. In order to not waste space, yet keep the amortized costs low, the insertion and deletion procedures should preserve two properties:

- the load factor of the dynamic table is bounded below by a positive constant, as well as above by 1, and
- the amortized cost of a table operation is bounded above by a constant.

The actual cost of each operation equals the number of elementary insertions or deletions.

You might think that if you double the table size upon inserting an item into a full table, then you should halve the size when deleting an item that would cause the table to become less than half full. This strategy does indeed guarantee that the load factor of the table never drops below $1/2$. Unfortunately, it can also cause the amortized cost of an operation to be quite large. Consider the following scenario. Perform n operations on a table T of size $n/2$, where n is an exact power of 2. The first $n/2$ operations are insertions, which by our previous analysis cost a total of $\Theta(n)$. At the end of this sequence of insertions, $T.num = T.size = n/2$. For the second $n/2$ operations, perform the following sequence:

insert, delete, delete, insert, insert, delete, delete, insert, insert, ...

The first insertion causes the table to expand to size n. The two deletions that follow cause the table to contract back to size $n/2$. Two further insertions cause another expansion, and so forth. The cost of each expansion and contraction is $\Theta(n)$, and there are $\Theta(n)$ of them. Thus, the total cost of the n operations is $\Theta(n^2)$, making the amortized cost of an operation $\Theta(n)$.

The problem with this strategy is that after the table expands, not enough deletions occur to pay for a contraction. Likewise, after the table contracts, not enough insertions take place to pay for an expansion.

How can we solve this problem? Allow the load factor of the table to drop below $1/2$. Specifically, continue to double the table size upon inserting an item into a full table, but halve the table size when deleting an item causes the table to become less than $1/4$ full, rather than $1/2$ full as before. The load factor of the table is therefore bounded below by the constant $1/4$, and the load factor is $1/2$ immediately after a contraction.

An expansion or contraction should exhaust all the built-up potential, so that immediately after expansion or contraction, when the load factor is $1/2$, the table's potential is 0. Figure 16.5 shows the idea. As the load factor deviates from $1/2$, the

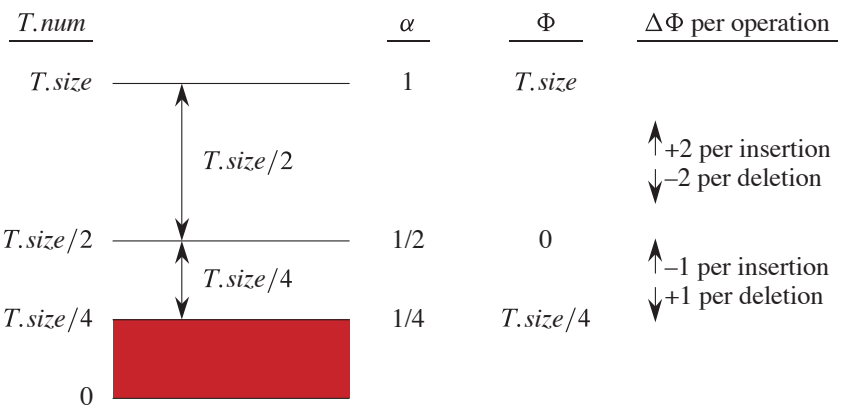

Figure 16.5 How to think about the potential function Φ for table insertion and deletion. When the load factor α is $1/2$, the potential is 0. In order to accumulate sufficient potential to pay for reinserting all $T.size$ items when the table fills, the potential needs to increase by 2 upon each insertion when $\alpha \geq 1/2$. Correspondingly, the potential decreases by 2 upon each deletion that leaves $\alpha \geq 1/2$. In order to accrue enough potential to cover the cost of reinserting all $T.size/4$ items when the table contracts, the potential needs to increase by 1 upon each deletion when $\alpha < 1/2$, and correspondingly the potential decreases by 1 upon each insertion that leaves $\alpha<1/2$. The red area represents load factors less than $1/4$, which are not allowed.

potential increases so that by the time an expansion or contraction occurs, the table has garnered sufficient potential to pay for copying all the items into the newly allocated table. Thus, the potential function should grow to $T.num$ by the time that the load factor has either increased to 1 or decreased to $1/4$. Immediately after either expanding or contracting the table, the load factor goes back to $1/2$ and the table's potential reduces back to 0.

We omit the code for TABLE-DELETE, since it is analogous to TABLE-INSERT. We assume that if a contraction occurs during TABLE-DELETE, it occurs after the item is deleted from the table. The analysis assumes that whenever the number of items in the table drops to 0, the table occupies no storage. That is, if $T.num = 0$, then $T.size = 0$.

How do we design a potential function that gives constant amortized time for both insertion and deletion? When the load factor is at least $1/2$, the same potential function, $\Phi(T) = 2(T.num - T.size/2)$, that we used for insertion still works. When the table is at least half full, each insertion increases the potential by 2 if the table does not expand, and each deletion reduces the potential by 2 if it does not cause the load factor to drop below $1/2$.

What about when the load factor is less than $1/2$, that is, when $1/4 \leq \alpha(T) < 1/2$? As before, when $\alpha(T) = 1/2$, so that $T.num = T.size/2$, the potential $\Phi(T)$ should be 0. To get the load factor from $1/2$ down to $1/4$, $T.size/4$ deletions need

to occur, at which time $T.num = T.size/4$. To pay for all the reinsertions, the potential must increase from 0 to $T.size/4$ over these $T.size/4$ deletions. Therefore, for each call of TABLE-DELETE until the table contracts, the potential should increase by

$$\frac{T.size/4}{T.size/4} = 1 .$$

Likewise, when $\alpha < 1/2$, each call of TABLE-INSERT should decrease the potential by 1. When $1/4 \le \alpha(T) < 1/2$, the potential function

$$\Phi(T) = T.size/2 - T.num$$

produces this desired behavior.

Putting the two cases together, we get the potential function

$$\Phi(T) = \begin{cases} 2(T.num - T.size/2) & \text{if } \alpha(T) \ge 1/2 , \\ T.size/2 - T.num & \text{if } \alpha(T) < 1/2 . \end{cases} \tag{16.5}$$

The potential of an empty table is 0 and the potential is never negative. Thus, the total amortized cost of a sequence of operations with respect to Φ provides an upper bound on the actual cost of the sequence. Figure 16.6 illustrates how the potential function behaves over a sequence of insertions and deletions.

Now, let's determine the amortized costs of each operation. As before, let num_i denote the number of items stored in the table after the ith operation, $size_i$ denote the total size of the table after the ith operation, $\alpha_i = num_i/size_i$ denote the load factor after the ith operation, Φ_i denote the potential after the ith operation, and $\Delta\Phi_i$ denote the change in potential due to the ith operation. Initially, $num_0 = 0$, $size_0 = 0$, and $\Phi_0 = 0$.

The cases in which the table does not expand or contract and the load factor does not cross $\alpha = 1/2$ are straightforward. As we have seen, if $\alpha_{i-1} \ge 1/2$ and the ith operation is an insertion that does not cause the table to expand, then $\Delta\Phi_i = 2$. Likewise, if the ith operation is a deletion and $\alpha_i \ge 1/2$, then $\Delta\Phi_i = -2$. Furthermore, if $\alpha_{i-1} < 1/2$ and the ith operation is a deletion that does not trigger a contraction, then $\Delta\Phi_i = 1$, and if the ith operation is an insertion and $\alpha_i < 1/2$, then $\Delta\Phi_i = -1$. In other words, if no expansion or contraction occurs and the load factor does not cross $\alpha = 1/2$, then

- if the load factor stays at or above $1/2$, then the potential increases by 2 for an insertion and decreases by 2 for a deletion, and
- if the load factor stays below $1/2$, then the potential increases by 1 for a deletion and decreases by 1 for an insertion.

In each of these cases, the actual cost c_i of the ith operation is just 1, and so